

Course Outline: Functions in JavaScript and TypeScript

Title: Functions in JavaScript and TypeScript **Prerequisite:** Control Flow and Basic TypeScript Syntax **Target Audience:** Beginner developers who understand variables, data types, and control structures **Total Lessons:** 24 **Estimated Total Length:** ~12,000 words **Format:** Mixed (46% hands-on)

Course Description

Functions are the fundamental building blocks of organized, reusable code. This course teaches you how to transform repetitive code into elegant, maintainable functions in both JavaScript and TypeScript. You'll learn to break down complex problems into smaller, manageable pieces by creating functions that accept input through parameters and produce output through return values.

By the end of this course, you'll understand the difference between functions, methods, and procedures. You'll write clean, self-documenting code that follows industry best practices like DRY (Don't Repeat Yourself) and Single Responsibility. Most importantly, you'll develop the mindset of a professional developer: thinking in terms of reusable components rather than one-off solutions.

This knowledge forms the foundation for everything that follows in your programming journey—from working with arrays and objects to building entire applications.

Course Philosophy

- This course emphasizes:
- **Single responsibility:** Each function should do one thing and do it well
 - **Pure functions:** Functions that work only with what they receive as parameters
 - **Code reusability:** Writing functions once and using them many times
 - **Readable code:** Function names that clearly communicate intent
 - **Incremental learning:** Building from simple blocks to complex function patterns
 - **Practical application:** Every concept practiced immediately with hands-on exercises

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome to Functions	1	~450
01 - Understanding Code Blocks and Functions	3	~1,650
02 - Creating and Naming Functions	4	~2,300
03 - Parameters and Arguments	4	~2,300
04 - Return Values and Function Types	4	~2,300
05 - Built-in Methods for Primitives	3	~1,650

Section	Lessons	Estimated Words
06 - Best Practices and Design Patterns	3	~1,650
07 - Assessment and Practice	2	~1,250
08 - Next Steps	1	~450
Total	24	~12,000

Section 00: Welcome to Functions

00.0 Welcome: Learning to Organize and Reuse Code  (~450 words)

Learning Objectives:

- Understand what functions are and why they're essential in programming
- Recognize how functions help organize code and eliminate repetition
- See the connection between functions and real-world problem-solving

Content Outline:

- The problem with repetitive code: copy-paste leads to bugs
- Real-world analogy: functions as recipes or instructions
- How functions transform messy code into organized, reusable components
- Overview of what you'll learn: from simple blocks to professional patterns
- Setting expectations: you'll write lots of functions, make mistakes, and improve

Transitions: This welcome sets the stage for understanding why functions matter. Next, you'll dive into the fundamental concept: code blocks.

Key Principles:

- Functions eliminate code duplication
- Well-named functions make code self-documenting
- Every professional programmer thinks in terms of functions

Examples:

- Comparison: 50 lines of repetitive code vs. 5 lines with a function
 - Real scenario: calculating tax on multiple purchases without and with functions
-

Section 01: Understanding Code Blocks and Functions

01.0 What is a Code Block?  (~500 words)

Learning Objectives:

- Define what a code block is in programming
- Identify code blocks in existing code
- Understand how curly braces create scope

Content Outline:

- Definition: A code block is a group of statements treated as a single unit
- Curly braces `{ }` define block boundaries
- Where you've already seen blocks: if statements, loops, switch cases
- Scope introduction: variables inside blocks vs. outside
- Indentation as visual organization

Transitions: You've been using code blocks in conditionals and loops. Now you'll learn how to give these blocks names and make them reusable—that's what functions are.

Key Principles:

- Code blocks group related statements together
- Blocks create their own scope for variables
- Proper indentation makes blocks readable

Examples:

- if statement block highlighting the curly braces
- for loop block showing where the block starts and ends
- Nested blocks: a loop inside an if statement

01.1 From Blocks to Functions: Giving Your Code a Name (~550 words)

Learning Objectives:

- Transform a code block into a named function
- Understand the syntax of function declaration
- Call a function to execute its code

Content Outline:

- The transition: from anonymous block to named function
- Basic function syntax: `function functionName() { }`
- Function declaration components: keyword, name, parentheses, body
- Calling/invoking a function: `functionName()`
- Execution flow: code jumps to function, executes, returns

Transitions: You've created your first function! Next, you'll practice writing and calling functions to build muscle memory.

Key Principles:

- Functions are named code blocks
- Function names should describe what the function does
- Functions don't run until you call them

Examples:

- Before: code block that prints a greeting
- After: function `greetUser()` that does the same

- Demonstrating calling the function multiple times

Hands-On Exercise: Create three simple functions:

1. `showWelcomeMessage()` - prints a welcome message
2. `displayCurrentDate()` - prints today's date
3. `calculateSum()` - calculates and prints $5 + 10$

Call each function twice to see them execute.

Expected Outcomes:

- Students create their first functions successfully
- Students understand the difference between defining and calling
- Students see how one function can be called multiple times

01.2 Your First Function: Declaration and Invocation (~600 words)

Learning Objectives:

- Write multiple functions in a single program
- Practice function naming conventions
- Debug common function mistakes (forgetting parentheses, typos)

Content Outline:

- Multiple functions in one file
- Function naming rules: camelCase, must start with letter
- Common mistakes: calling without `()`, misspelling function name
- The order doesn't matter for declarations (hoisting preview)
- Using `console.log` inside functions for visibility

Transitions: You can create functions, but they're not very flexible yet—they do the same thing every time. The next section introduces parameters, which let you pass different data to your functions.

Key Principles:

- Functions can be defined in any order
- Function calls need parentheses `()`
- `Console.log` is your debugging friend

Examples:

- Program with 3 functions called in sequence
- Demonstrating error when forgetting parentheses
- Showing how to use `console.log` to trace execution

Hands-On Exercise: Debug a program with 5 functions that have various errors:

- Missing parentheses on function call
- Misspelled function name
- Function called before it's needed

- Add `console.log` to trace execution order

Create two new functions:

- `showMenu()` - displays a restaurant menu
- `showReceipt()` - displays a mock receipt

Expected Outcomes:

- Students can identify and fix common function errors
 - Students write clean, working functions
 - Students understand execution flow through multiple functions
-

Section 02: Creating and Naming Functions

02.0 Function Declaration Syntax in TypeScript 📖 (~500 words)

Learning Objectives:

- Write proper function syntax in TypeScript
- Understand the parts of a function declaration
- Recognize why explicit syntax helps prevent errors

Content Outline:

- Complete function anatomy: `function name(): returnType { body }`
- TypeScript's stricter requirements vs JavaScript
- The `void` return type for functions that don't return values
- Why TypeScript requires return type annotations
- Function declaration vs. function definition

Transitions: Now that you understand the formal syntax, you need to learn the art of naming functions—one of the most important skills in programming.

Key Principles:

- TypeScript requires explicit return types
- `void` means "this function doesn't return a value"
- Proper syntax prevents runtime errors

Examples:

- Valid TypeScript function with void return
- JavaScript function vs. TypeScript function syntax
- Common syntax errors and how TypeScript catches them

02.1 Naming Functions: Making Code Self-Documenting 📖 (~600 words)

Learning Objectives:

- Write descriptive, self-documenting function names

- Follow naming conventions for different function purposes
- Avoid vague or misleading names

Content Outline:

- Verb-based naming: functions DO things (calculateTotal, validateEmail)
- Avoiding abbreviations and single letters
- Boolean-returning functions: use **is**, **has**, **can** (isValid, hasPermission)
- Length vs. clarity: better too long than too vague
- Consistency: similar functions should follow similar patterns

Transitions: You've learned to name functions well. But there's more than one way to create a function in TypeScript. Next, you'll explore alternative syntaxes.

Key Principles:

- Function names should clearly state what the function does
- Use verbs for actions, predicates for booleans
- Consistency makes code predictable

Examples:

- Good names: **calculateSalesTax**, **isEligibleForDiscount**, **formatCurrency**
- Bad names: **doStuff**, **calc**, **x**, **processData**
- Refactoring poorly named functions

Hands-On Exercise: Review 10 poorly named functions and rename them:

- **func1()** → should check if number is even
- **x()** → should convert Celsius to Fahrenheit
- **doIt()** → should validate email format

Write 3 new functions with excellent names that:

1. Check if a student passed (score >= 60)
2. Calculate area of a rectangle
3. Display a formatted address

Expected Outcomes:

- Students write clear, descriptive function names
- Students can evaluate and improve existing function names
- Students understand the impact of naming on code readability

02.2 Function Expressions and Arrow Functions 📖 (~550 words)

Learning Objectives:

- Understand different ways to create functions in TypeScript
- Compare function declarations, expressions, and arrow functions
- Recognize when each syntax is appropriate

Content Outline:

- Function declarations (what you've been using)
- Function expressions: `const myFunction = function() { }`
- Arrow functions: `const myFunction = () => { }`
- Arrow function shorthand for single expressions
- When to use each style (declarations for hoisting, arrows for callbacks)
- TypeScript type annotations with each style

Transitions: Now that you know multiple ways to write functions, practice using each style to build familiarity.

Key Principles:

- All three styles create functions
- Arrow functions are more concise
- Choose the style that fits your use case

Examples:

- Same function written three ways
- Arrow function with implicit return: `const double = (x: number): number => x * 2`
- Multi-line arrow function with explicit return

02.3 Practice: Writing Functions in Different Styles (~650 words)

Learning Objectives:

- Write the same function using different syntaxes
- Choose appropriate syntax for different scenarios
- Convert between function styles fluently

Content Outline:

- Converting function declarations to expressions
- Converting expressions to arrow functions
- When implicit return works vs. when you need explicit
- Practicing with TypeScript type annotations for each style
- Refactoring existing code to use different function styles

Transitions: You can now create functions in multiple styles. But functions become truly powerful when they can work with different data. The next section introduces parameters.

Key Principles:

- Fluency with all function styles makes you versatile
- Arrow functions excel in callbacks (you'll see this later)
- Consistency within a project matters more than which style you choose

Examples:

- Temperature converter written three ways
- Set of utility functions using different styles appropriately

- Refactoring a program to use consistent style

Hands-On Exercise: Convert 5 function declarations to arrow functions:

1. A greeting function
2. A function that checks if number is positive
3. A function that calculates tip amount
4. A function that formats a name
5. A function that converts miles to kilometers

Create 3 new functions using arrow syntax:

- One that returns a boolean
- One that returns a number
- One that returns a string

Expected Outcomes:

- Students can write functions in all three styles
 - Students understand the equivalence between styles
 - Students can choose appropriate syntax for context
-

Section 03: Parameters and Arguments

03.0 Understanding Parameters: Giving Data to Functions 📖 (~500 words)

Learning Objectives:

- Define what parameters and arguments are
- Understand how data flows into functions
- Recognize the difference between parameters and arguments

Content Outline:

- Parameters: placeholders in function definition
- Arguments: actual values passed when calling
- The terminology: parameters in declaration, arguments in invocation
- How parameters make functions flexible
- Parameters create local variables inside the function

Transitions: You understand the concept—now it's time to write functions that accept data and use it.

Key Principles:

- Parameters make functions reusable with different data
- Parameters are variables that exist only inside the function
- Arguments are the actual values you pass

Examples:

- Function without parameters (always does same thing)

- Same function with parameter (can work with any value)
- Visual: parameter as blueprint, argument as concrete value

03.1 Working with Multiple Parameters 📖 (~600 words)

Learning Objectives:

- Create functions that accept multiple parameters
- Pass multiple arguments when calling functions
- Understand parameter order matters

Content Outline:

- Syntax for multiple parameters: `function name(param1: type, param2: type)`
- Calling with multiple arguments: order is critical
- Using multiple parameters together in function body
- Descriptive parameter names improve clarity
- Common mistakes: wrong number of arguments, wrong order

Transitions: You can work with multiple parameters now. TypeScript adds an extra layer of safety through type annotations—let's explore that next.

Key Principles:

- Parameter order in definition matches argument order in call
- Descriptive parameter names are self-documenting
- Each parameter can have a different type

Examples:

- `calculateRectangleArea(width: number, height: number)`
- `formatFullName(firstName: string, lastName: string)`
- Demonstration of what happens when arguments are in wrong order

Hands-On Exercise: Create functions with multiple parameters:

1. `calculateTotal(price: number, quantity: number, taxRate: number)` - returns total cost
2. `buildGreeting(greeting: string, name: string, punctuation: string)` - returns formatted greeting
3. `isInRange(value: number, min: number, max: number)` - returns boolean

Test each function with different arguments.

Expected Outcomes:

- Students write functions accepting 2-3 parameters
- Students understand the importance of parameter order
- Students use meaningful parameter names

03.2 Type Annotations for Parameters in TypeScript 📖 (~550 words)

Learning Objectives:

- Write proper type annotations for function parameters
- Understand how TypeScript prevents type-related errors
- Recognize the benefits of explicit typing

Content Outline:

- Required syntax: `parameter: type`
- How TypeScript validates argument types at compile time
- Common types for parameters: number, string, boolean
- What happens when you pass wrong type
- Benefits: catch errors before running code, better IDE support

Transitions: Type safety makes your functions more reliable. Now let's practice writing type-safe functions that solve real problems.

Key Principles:

- Every parameter should have a type annotation
- TypeScript prevents you from passing wrong types
- Explicit types make code self-documenting

Examples:

- Function with typed parameters
- Attempting to pass wrong type (TypeScript error)
- IDE autocomplete benefits from type information

03.3 Practice: Building Functions that Accept Input (~650 words)

Learning Objectives:

- Build a collection of utility functions with parameters
- Practice parameter typing and naming
- Debug parameter-related errors

Content Outline:

- Creating math utility functions (add, subtract, multiply, divide)
- Creating string utility functions (capitalize, repeat, truncate)
- Creating validation functions (isEmail, isPhoneNumber, isPositive)
- Debugging: missing type annotations, wrong types, parameter order issues
- Testing functions with edge cases

Transitions: Your functions can now accept input, but they can't return results yet. The next section teaches you how functions communicate results back to the caller.

Key Principles:

- Start simple, then add complexity
- Test with various inputs, including edge cases
- `Console.log` helps verify parameter values

Examples:

- Complete set of string manipulation functions
- Math utilities with proper typing
- Validation functions returning booleans

Hands-On Exercise: Build a utility function library:

Math utilities:

- `getMax(a: number, b: number)` - returns larger number
- `calculateAverage(num1: number, num2: number, num3: number)` - returns average
- `isEven(num: number)` - returns boolean

String utilities:

- `repeatString(text: string, times: number)` - returns repeated text
- `getInitials(firstName: string, lastName: string)` - returns initials
- `truncate(text: string, maxLength: number)` - returns truncated text

Test each function with at least 3 different inputs.

Expected Outcomes:

- Students create a working utility library
 - Students practice parameter typing consistently
 - Students test functions thoroughly
-

Section 04: Return Values and Function Types

04.0 How Functions Return Values 📖 (~500 words)

Learning Objectives:

- Understand what return values are and why they matter
- Write the return statement correctly
- Recognize that return exits the function immediately

Content Outline:

- The purpose of return: sending data back to caller
- Return statement syntax: `return value;`
- Return ends function execution immediately
- Capturing returned values in variables
- Functions without return implicitly return undefined

Transitions: Understanding return conceptually is the first step. Next, you'll practice writing functions that calculate and return results.

Key Principles:

- Return sends a value back to wherever the function was called

- Code after return never executes
- Returned values can be stored, used, or ignored

Examples:

- Function that returns a calculated value
- Demonstrating code after return doesn't run
- Using a return value in an expression

04.1 Return Type Annotations in TypeScript 📖 (~600 words)

Learning Objectives:

- Annotate return types in TypeScript functions
- Understand how TypeScript validates return values
- Practice with different return types (number, string, boolean)

Content Outline:

- Return type syntax: `function name(): returnType`
- TypeScript verifies you return the promised type
- Common return types: number, string, boolean, void
- What happens when return type doesn't match
- Why explicit return types improve code quality

Transitions: You can now specify and return typed values. But not all functions need to return values—let's explore that distinction.

Key Principles:

- Always specify return type in TypeScript
- TypeScript catches return type mismatches
- Return type documentation helps users of your function

Examples:

- Function returning number with annotation
- TypeScript error when returning wrong type
- Multiple functions with different return types

Hands-On Exercise: Create typed functions that return values:

1. `calculateCircleArea(radius: number): number` - returns area
2. `isAdult(age: number): boolean` - returns true if age ≥ 18
3. `formatPrice(amount: number): string` - returns "\$X.XX" format
4. `getGrade(score: number): string` - returns letter grade

Store each return value in a variable and use it.

Expected Outcomes:

- Students write proper return type annotations

- Students understand type checking on returns
- Students use returned values effectively

04.2 Functions vs Procedures: Return vs Side Effects (~550 words)

Learning Objectives:

- Distinguish between functions (return values) and procedures (side effects)
- Understand when to use return vs. console.log
- Recognize methods as functions attached to objects

Content Outline:

- Functions: do work and return results
- Procedures: do work without returning (void return type)
- Side effects: console.log, modifying global state, etc.
- Methods: functions that belong to objects/types
- Best practice: prefer returning values over side effects

Transitions: Understanding this distinction helps you design better functions. Now let's practice creating both types and knowing when to use each.

Key Principles:

- Functions that return are more flexible and testable
- Procedures (void functions) are for actions without results
- Methods are just functions with special context

Examples:

- calculateTax (function - returns value)
- displayWelcome (procedure - void, uses console.log)
- String methods like .toUpperCase() as examples

04.3 Practice: Functions that Calculate and Return (~650 words)

Learning Objectives:

- Build a library of calculation functions
- Practice using return values in expressions
- Combine multiple function calls

Content Outline:

- Creating calculation functions (financial, geometric, statistical)
- Using return values in further calculations
- Chaining function calls
- Return early pattern for validation
- Debugging return value issues

Transitions: You've mastered creating your own functions with parameters and returns. But JavaScript and TypeScript also provide built-in methods for primitive types—let's explore those next.

Key Principles:

- Test functions by using their return values
- Return early for cleaner code
- One function can use another's return value

Examples:

- Financial calculator functions working together
- Using return value in conditional
- Chained calculations

Hands-On Exercise: Create a complete calculation library:

Financial:

- `calculateTip(billAmount: number, tipPercent: number): number`
- `calculateDiscount(price: number, discountPercent: number): number`
- `calculateFinalPrice(price: number, taxRate: number): number`

Geometric:

- `calculateRectanglePerimeter(width: number, height: number): number`
- `calculateCircleCircumference(radius: number): number`
- `calculateTriangleArea(base: number, height: number): number`

Then create a program that:

1. Calculates bill with tip
2. Applies discount
3. Adds tax
4. Shows final amount

Expected Outcomes:

- Students create practical calculation functions
- Students chain function calls effectively
- Students understand return value flow

Section 05: Built-in Methods for Primitives

05.0 Understanding Methods vs Functions (~500 words)

Learning Objectives:

- Define what methods are in programming
- Understand method syntax: `value.method()`
- Recognize built-in methods for string, number, and other primitives

Content Outline:

- Methods are functions attached to values/objects

- Dot notation syntax for calling methods
- Difference between standalone functions and methods
- Every primitive type has built-in methods
- Methods often return transformed values (they don't mutate primitives)

Transitions: Now that you understand what methods are, let's explore the most commonly used ones for strings—you'll use these constantly in your programming career.

Key Principles:

- Methods belong to specific types
- Primitive values have methods even though they're not objects
- Most primitive methods return new values (immutability)

Examples:

- `"hello".toUpperCase()` vs `toUpperCase("hello")`
- Demonstrating method calls on different types
- Showing that originals don't change

05.1 String Methods: Manipulating Text 📖 (~600 words)

Learning Objectives:

- Use essential string methods fluently
- Chain string methods together
- Apply string methods to solve practical problems

Content Outline:

- Case transformation: `.toLowerCase()`, `.toUpperCase()`
- Trimming whitespace: `.trim()`, `.trimStart()`, `.trimEnd()`
- Searching: `.includes()`, `.indexOf()`, `.startsWith()`, `.endsWith()`
- Extracting: `.substring()`, `.slice()`, `.charAt()`
- Replacing: `.replace()`, `.replaceAll()`
- Splitting and joining: `.split()` (and `Array.join()`)

Transitions: String methods are essential tools. Next, you'll explore number methods and the Math object for numerical operations.

Key Principles:

- String methods don't modify the original string
- Methods can be chained: `text.trim().toLowerCase()`
- Return values are often different types (e.g., `includes` returns boolean)

Examples:

- Email validation using string methods
- Formatting user input (trim and case normalize)
- Parsing text data

Hands-On Exercise: Create utility functions using string methods:

1. `cleanInput(text: string): string` - trims and converts to lowercase
2. `validateEmail(email: string): boolean` - checks for @ and .
3. `extractUsername(email: string): string` - gets part before @
4. `maskPassword(password: string): string` - replaces all but last 4 chars with *
5. `isValidPhoneNumber(phone: string): boolean` - checks format

Test each with various inputs including edge cases.

Expected Outcomes:

- Students use string methods confidently
- Students chain methods appropriately
- Students solve practical string problems

05.2 Number Methods and Math Functions 📖 (~550 words)

Learning Objectives:

- Use number methods and Math object functions
- Round, floor, and ceil numbers appropriately
- Generate random numbers and perform common calculations

Content Outline:

- Number methods: `.toFixed()`, `.toPrecision()`, `.toString()`
- Math object: `Math.round()`, `Math.floor()`, `Math.ceil()`
- Rounding to specific decimal places
- `Math.random()` for random numbers
- Useful Math functions: `Math.abs()`, `Math.max()`, `Math.min()`, `Math.pow()`, `Math.sqrt()`
- Converting strings to numbers: `Number()`, `parseInt()`, `parseFloat()`

Transitions: You now have a solid foundation in both custom functions and built-in methods. The next section teaches you professional patterns and best practices that separate beginner code from production-quality code.

Key Principles:

- Math is a built-in object with utility functions
- Random numbers need scaling to useful ranges
- Choose appropriate rounding method for your use case

Examples:

- Formatting currency with `.toFixed(2)`
- Generating random integers in a range
- Using `Math.max` to find largest value

Hands-On Exercise: Create number utility functions:

1. `formatCurrency(amount: number): string` - returns \$X.XX format

2. `getRandomInt(min: number, max: number): number` - returns random integer in range
3. `roundToDecimal(num: number, places: number): number` - rounds to specific decimal places
4. `getPercentage(part: number, whole: number): number` - calculates percentage
5. `clamp(value: number, min: number, max: number): number` - constrains value to range

Build a simple tip calculator using these utilities.

Expected Outcomes:

- Students use Math functions correctly
 - Students format numbers appropriately
 - Students combine functions to solve problems
-

Section 06: Best Practices and Design Patterns

06.0 DRY Principle and Pure Functions 📖 (~550 words)

Learning Objectives:

- Understand and apply the DRY (Don't Repeat Yourself) principle
- Define and write pure functions
- Recognize when to extract code into a function

Content Outline:

- DRY: if you write it twice, make it a function
- Identifying repetitive code patterns
- Pure functions: same inputs always produce same outputs
- Pure functions have no side effects
- Benefits: predictable, testable, reusable
- Avoiding global variable dependencies in pure functions

Transitions: DRY and pure functions are foundational principles. Next, you'll learn specific patterns that make your functions cleaner and more maintainable.

Key Principles:

- Repetition is a signal to create a function
- Pure functions are easier to test and debug
- Functions should work with parameters, not global variables

Examples:

- Before/after: repetitive code extracted into function
- Pure function example: `add(a, b)` always returns same result
- Impure function example: function using `Math.random()` or global state

06.1 Single Responsibility and Return Early Pattern 📖 (~600 words)

Learning Objectives:

- Apply the Single Responsibility Principle to functions
- Use the Return Early pattern for cleaner code
- Refactor functions to improve clarity

Content Outline:

- Single Responsibility: one function, one job
- How to identify when a function does too much
- Breaking large functions into smaller ones
- Return Early (guard clauses): validate inputs first, then main logic
- Benefits: less nesting, easier to read
- Practical refactoring examples

Transitions: You've learned positive patterns. Now let's look at common mistakes (anti-patterns) to avoid.

Key Principles:

- Functions should do one thing well
- Return early to avoid deep nesting
- Guard clauses improve readability

Examples:

- God Function split into specific functions
- Before/after: nested conditions vs. return early
- Real-world: validation function using guard clauses

Hands-On Exercise: Refactor poorly designed functions:

1. A "God Function" that validates user, calculates price, and sends email - split into 3 functions
2. A deeply nested function - refactor using return early pattern
3. A function doing multiple unrelated tasks - separate responsibilities

Create new functions following best practices:

- `validateAge(age: number): boolean` - uses guard clauses
- `calculateShippingCost(weight: number, distance: number): number` - single responsibility
- `formatUserName(user: {firstName: string, lastName: string}): string` - pure function

Expected Outcomes:

- Students recognize when functions are too complex
- Students apply return early pattern
- Students write focused, single-purpose functions

06.2 Anti-Patterns: God Functions and Global Dependencies 📖 (~500 words)

Learning Objectives:

- Identify God Functions (functions that do too much)
- Recognize global dependency anti-pattern
- Refactor anti-patterns into clean code

Content Outline:

- God Function: a function that calculates, validates, saves, and sends notifications
- Why God Functions are problematic: hard to test, maintain, reuse
- Global Dependencies: functions that access variables outside their scope
- Why global dependencies break: harder to test, unpredictable behavior
- How to fix: pass everything as parameters
- Real examples of anti-patterns in the wild

Transitions: Understanding what NOT to do is as important as knowing best practices. Now you're ready to demonstrate your knowledge through assessment.

Key Principles:

- Functions doing multiple jobs are hard to maintain
- Global variables make functions unpredictable
- Pass data explicitly through parameters

Examples:

- God Function example with refactoring
- Global dependency with before/after refactor
- Real-world code smell examples

Anti-Pattern Examples:

```
// God Function - doing too much
function processOrder() {
  // validates user
  // calculates price
  // applies discount
  // charges card
  // sends email
  // updates inventory
  // logs activity
}

// Global Dependency - accessing outside scope
let taxRate = 0.08;
function calculateTotal(price: number): number {
  return price * (1 + taxRate); // depends on global!
}

// Better: pure function
function calculateTotal(price: number, taxRate: number): number {
  return price * (1 + taxRate);
}
```

Section 07: Assessment and Practice

07.0 Comprehensive Function Practice Challenge 📖 (~650 words)

Learning Objectives:

- Apply all function concepts in a comprehensive project
- Demonstrate understanding of parameters, returns, and best practices
- Build a complete utility library following professional patterns

Content Outline:

- Building a real-world utility library
- Organizing functions by category
- Documenting functions with comments
- Testing all functions thoroughly
- Following naming conventions and best practices

Transitions: You've completed the practice challenge. Now let's assess your understanding of functions through a comprehensive knowledge check.

Key Principles:

- Real projects combine multiple concepts
- Organization and naming matter
- Thorough testing catches bugs early

Examples:

- Complete utility library structure
- Documentation examples
- Testing strategy for functions

Hands-On Exercise: Build a comprehensive Personal Finance Calculator library with these functions:

Income Calculations:

- `calculateHourlyWage(annualSalary: number): number`
- `calculateAnnualIncome(hourlyWage: number, hoursPerWeek: number): number`
- `calculateNetIncome(grossIncome: number, taxRate: number): number`

Budget Calculations:

- `calculateMonthlyBudget(annualIncome: number): number`
- `getCategoryBudget(monthlyBudget: number, categoryPercent: number): number`
- `isOverBudget(spent: number, budgeted: number): boolean`

Savings Calculations:

- `calculateSavingsGoal(monthlyIncome: number, savingsRate: number): number`
- `getMonthsToGoal(goalAmount: number, monthlySavings: number): number`
- `calculateCompoundInterest(principal: number, rate: number, years: number): number`

Utilities:

- `formatCurrency(amount: number): string`
- `validatePositiveNumber(num: number): boolean`
- `roundToTwoDecimals(num: number): number`

Create a main program that uses all functions to:

1. Calculate someone's annual and monthly income
2. Plan their budget across categories
3. Calculate savings goals
4. Show formatted output

Success Criteria:

- All functions have proper type annotations
- All functions follow single responsibility
- All functions use descriptive names
- All calculations are accurate
- Code is well-organized and readable

Expected Outcomes:

- Students build a complete, working application
- Students demonstrate mastery of all function concepts
- Students follow professional coding practices

07.1 Functions Knowledge Check 🧐 (~600 words)

Learning Objectives:

- Evaluate understanding of function concepts
- Demonstrate ability to analyze and write functions
- Apply best practices in practical scenarios

Content Outline: This assessment uses scenario-based questions to evaluate your understanding of functions. Each scenario presents a real programming challenge.

Question Format: Scenario-based questions (7 questions)

Assessment Questions:

Question 1: Function Declaration (10 points) Scenario: You need to create a function that calculates the area of a rectangle.

Which function declaration is correct in TypeScript?

- a) `function calculateArea(width, height) { return width * height; }` b) `function calculateArea(width: number, height: number): number { return width * height; }` c) `function calculateArea(number width, number height): number { return width * height; }` d) `calculateArea(width: number, height: number): number => width * height`

Correct Answer: b **Explanation:** TypeScript requires type annotations for parameters and return type.

Question 2: Pure Functions (15 points) Scenario: Your team lead asks you to write a pure function for calculating sales tax.

Which of these is a pure function?

a)

```
let taxRate = 0.08;
function calculateTax(amount: number): number {
  return amount * taxRate;
}
```

b)

```
function calculateTax(amount: number): number {
  console.log("Calculating tax...");
  return amount * 0.08;
}
```

c)

```
function calculateTax(amount: number, taxRate: number): number {
  return amount * taxRate;
}
```

d)

```
function calculateTax(amount: number): number {
  return amount * Math.random();
}
```

Correct Answer: c **Explanation:** A pure function depends only on its parameters and has no side effects. Option a uses global variable, b has side effect (console.log), d is non-deterministic (random).

Question 3: Return Early Pattern (15 points) Scenario: You're refactoring a function that validates user age for account creation.

Which implementation uses the Return Early pattern correctly?

a)

```
function validateAge(age: number): boolean {
  if (age >= 18) {
```

```
    if (age <= 120) {  
        return true;  
    } else {  
        return false;  
    }  
} else {  
    return false;  
}  
}
```

b)

```
function validateAge(age: number): boolean {  
    if (age < 18) return false;  
    if (age > 120) return false;  
    return true;  
}
```

c)

```
function validateAge(age: number): boolean {  
    let isValid = false;  
    if (age >= 18 && age <= 120) {  
        isValid = true;  
    }  
    return isValid;  
}
```

Correct Answer: b **Explanation:** Return Early uses guard clauses to exit immediately on invalid conditions, avoiding nesting.

Question 4: Single Responsibility (15 points) Scenario: You're reviewing code and find this function. What's wrong with it?

```
function processUser(email: string, password: string): string {  
    // Validates email format  
    if (!email.includes("@")) return "Invalid email";  
  
    // Validates password strength  
    if (password.length < 8) return "Password too short";  
  
    // Saves to database  
    saveToDatabase(email, password);  
  
    // Sends welcome email  
    sendWelcomeEmail(email);  
}
```

```
// Logs activity
console.log("User created:", email);

return "Success";
}
```

What principle does this violate?

a) DRY (Don't Repeat Yourself) b) Single Responsibility Principle c) Return Early Pattern d) Type Safety

Correct Answer: b **Explanation:** This is a God Function doing validation, database operations, email sending, and logging. It should be split into focused functions.

Question 5: Parameter vs Argument (10 points) Scenario: You're explaining code to a junior developer.

In this code, identify parameters and arguments:

```
function greet(name: string, time: string): string {
    return `Good ${time}, ${name}!`;
}

const message = greet("Alice", "morning");
```

Which statement is correct?

a) "Alice" and "morning" are parameters; name and time are arguments b) name and time are parameters; "Alice" and "morning" are arguments c) All four are parameters d) All four are arguments

Correct Answer: b **Explanation:** Parameters are in function definition (name, time), arguments are actual values passed ("Alice", "morning").

Question 6: Method vs Function (15 points) Scenario: You need to convert a string to uppercase.

Which approach is correct and why?

a) `const result = toUpperCase("hello");` - calling a function b) `const result = "hello".toUpperCase();` - calling a method c) Both are correct and identical d) Neither will work in TypeScript

Correct Answer: b **Explanation:** `toUpperCase` is a method on strings, called with dot notation. There's no standalone `toUpperCase` function.

Question 7: Practical Application (20 points) Scenario: You're building a shopping cart. You need a function to calculate the final price after discount and tax.

Which implementation follows best practices?

a)

```
function calculatePrice(price: number): number {  
    let discount = 0.1;  
    let tax = 0.08;  
    return price * (1 - discount) * (1 + tax);  
}
```

b)

```
function calculatePrice(price: number, discountRate: number, taxRate: number):  
number {  
    const discountedPrice = price * (1 - discountRate);  
    const finalPrice = discountedPrice * (1 + taxRate);  
    return finalPrice;  
}
```

c)

```
function calculatePrice(price, discount, tax) {  
    return price * (1 - discount) * (1 + tax);  
}
```

Correct Answer: b **Explanation:** Best implementation: proper TypeScript types, parameters instead of hardcoded values, clear intermediate variables for readability, pure function design.

Evaluation Criteria:

- 90-100 points: Excellent understanding of functions
- 75-89 points: Good grasp, review weak areas
- 60-74 points: Adequate, needs more practice
- Below 60: Review course material and retry

Score Interpretation:

- **Excellent (90-100):** You understand functions deeply and can apply best practices
 - **Good (75-89):** Solid foundation, review specific concepts you missed
 - **Adequate (60-74):** Core understanding present, needs reinforcement through practice
 - **Needs Review (< 60):** Revisit lessons and complete additional exercises
-

Section 08: Next Steps

08.0 Your Journey with Functions Continues (~450 words)

Content Outline:**Congratulations on Completing Functions in JavaScript and TypeScript!**

You've achieved a crucial milestone in your programming journey. Functions are the building blocks of every application you'll ever create, and you now have the skills to write clean, reusable, maintainable code.

What You've Mastered:

You can now create functions from scratch, understanding exactly what code blocks are and how to give them names. You've learned three different ways to write functions in TypeScript—declarations, expressions, and arrow functions—and you know when to use each style.

You've mastered parameters and arguments, making your functions flexible enough to work with any data. You understand type annotations in TypeScript and why they make your code safer and more predictable. You can write functions that return values and functions that perform actions, knowing exactly when to use each approach.

Most importantly, you've learned to think like a professional developer. You apply the DRY principle to eliminate repetition. You write pure functions that are predictable and testable. You follow the Single Responsibility Principle, creating focused functions that do one thing well. You've learned to recognize and avoid anti-patterns like God Functions and global dependencies.

How This Connects to Your Future:

Everything you learn from this point forward builds on functions. When you work with arrays next, you'll use functions like `map`, `filter`, and `reduce`. When you create objects, you'll write methods—which are just functions attached to objects. When you build web applications, you'll write functions to handle user interactions, process data, and update interfaces.

Functions are universal. Whether you're building websites, mobile apps, or backend services, you'll spend your days writing and calling functions. The patterns you've learned here—pure functions, single responsibility, return early—are patterns you'll use every single day as a professional developer.

Your Next Steps:

Continue to the next course where you'll learn about arrays and collections. You'll discover powerful array methods that use functions as arguments—everything you've learned about functions will make that content much easier to understand.

Keep practicing. The Mastering JS learnpack continues to grow with you, introducing progressively complex challenges that require all your function skills. Every exercise reinforces what you've learned here.

Final Encouragement:

Writing good functions is an art that improves with practice. Don't worry if your first functions aren't perfect—refactoring is part of the process. Every function you write makes you better at organizing code, naming things clearly, and thinking in terms of reusable components.

You're not just learning to code—you're learning to think like a programmer. Keep building, keep practicing, and remember: every complex program is just a collection of simple functions working together.

Welcome to the world of functional thinking. Your code will never be the same.
